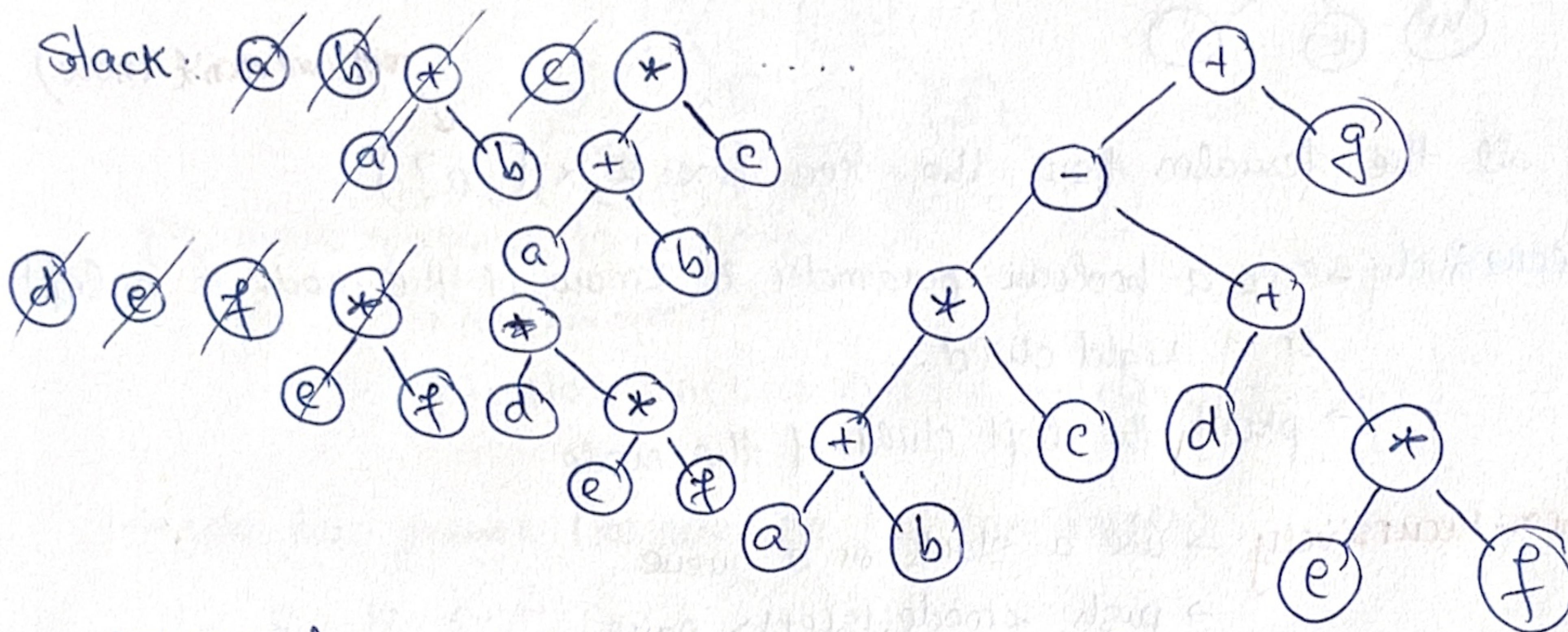


1. Build a binary tree for an arithmetic expression that contains the operators $+$, $-$, $*$, $/$. Use the postfix notation as input

$$\text{Ex: } (a+b)*c - (d+e*f) + g \Rightarrow ab+c*def*+-g+$$



$a = \text{operand} \Rightarrow \text{push to stack}$

$+ = \text{operator} \Rightarrow \text{pop last two from the stack}$

Stack operations: $\text{init}(s)$
 $\text{push}(s, e)$
 $\text{pop}(s, e)$
 $\text{isEmpty}(s)$
 $\text{top}(s)$

Representation:

Node:

info: TElem

$\text{left, right: } \uparrow \text{Node}$

BT:

$\text{root: } \uparrow \text{Node}$

subalgorithm $\text{buildTree}(\text{postfix})$:

$\text{init}(s)$

for e in postfix:

if e is an operand

$\text{node} \leftarrow \text{allocate}()$

$[\text{node}].\text{info} \leftarrow e$

$[\text{node}].\text{left} \leftarrow \text{Nil}$

$[\text{node}].\text{right} \leftarrow \text{Nil}$

$\text{push}(s, \text{node})$

else

$e_1 \leftarrow \text{pop}(s)$

$e_2 \leftarrow \text{pop}(s)$

$\text{node} \leftarrow \text{allocate}()$

:

$[\text{node}].\text{info} \leftarrow e$

$[\text{node}].\text{left} \leftarrow e_2$

$[\text{node}].\text{right} \leftarrow e_1$

$\text{push}(s, \text{node})$

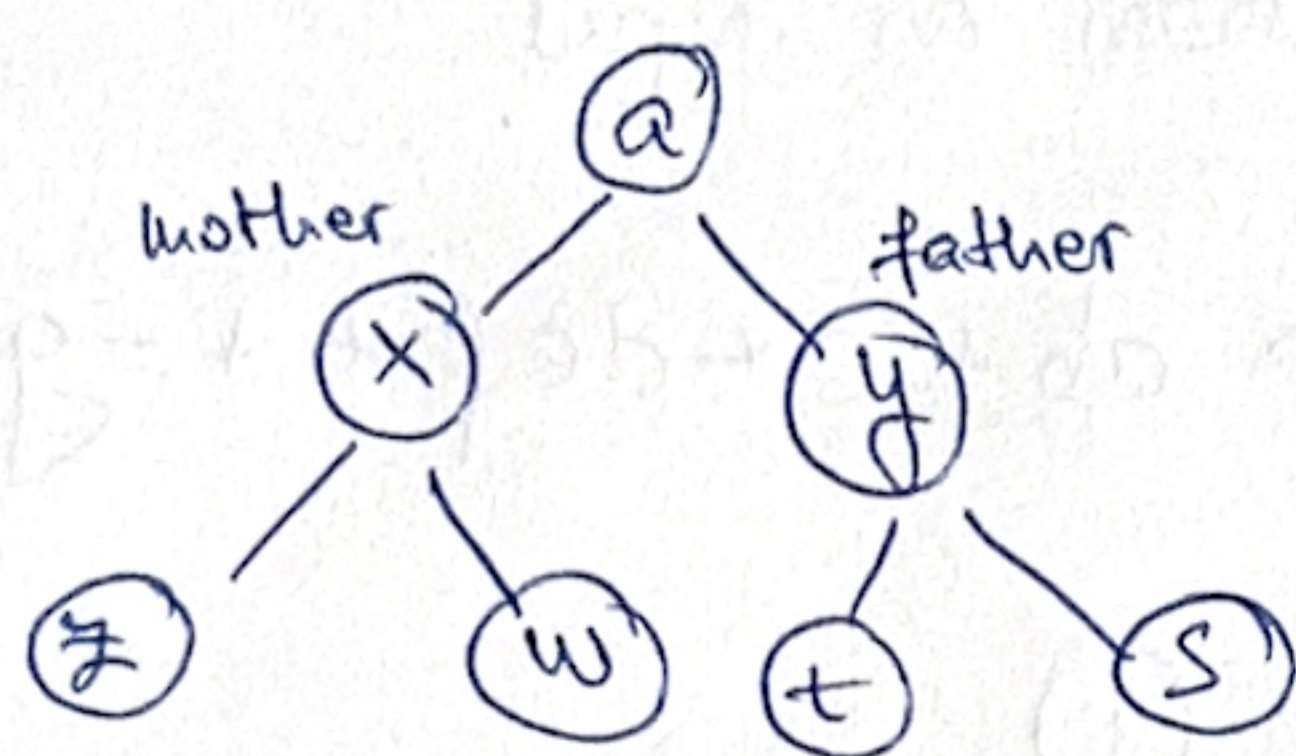
$\text{init}(bt)$ // container has a constructor of its own.

$bt.\text{root} \leftarrow \text{pop}(s)$

$\text{buildTree} \leftarrow bt$

2) We are given a BT that represents the ancestors of a person up to the m th generation, where the left subtree represents the maternal line and the right subtree represents the paternal line.

EX:



root (we don't know)

a) print all the females from the tree. (ex: z, x, t, a?)

- **recursively** → keep a boolean parameter to know if the node is a left or a right child.

→ print the left child of the node

- **non-recursively** → use a stack or a queue
→ push <node, isleft> pairs

← print left child approach

• subalg printFemales(bt):

init(q)

push(q, bt.root)

while not isEmpty(q) execute:

current ← pop(q)

print([current].left.info)

push(q, [current].left)

push(q, [current].right)

if bt.root ≠ NIL then

if [current].left ≠ NIL then

By convention, we never push NIL nodes to the queue.

if [current].right ≠ NIL then

- If you consider the root to be the female, print([bt.root].info) (*)

- Complete BT → check once for both children. (≠ NIL)

b) display all ancestors of degree K. (ex: K=0)

K=0 ⇒ a

K=1 ⇒ x, y

K=2 ⇒ z, w, t, s

- **recursively** → add a parameter for current lev.

→ decrement the value of K

(print current node when K=0)

- **non-recursively** → use a stack / queue in which we push <node, level> pairs. Print the info when level=K.

→ use a queue:

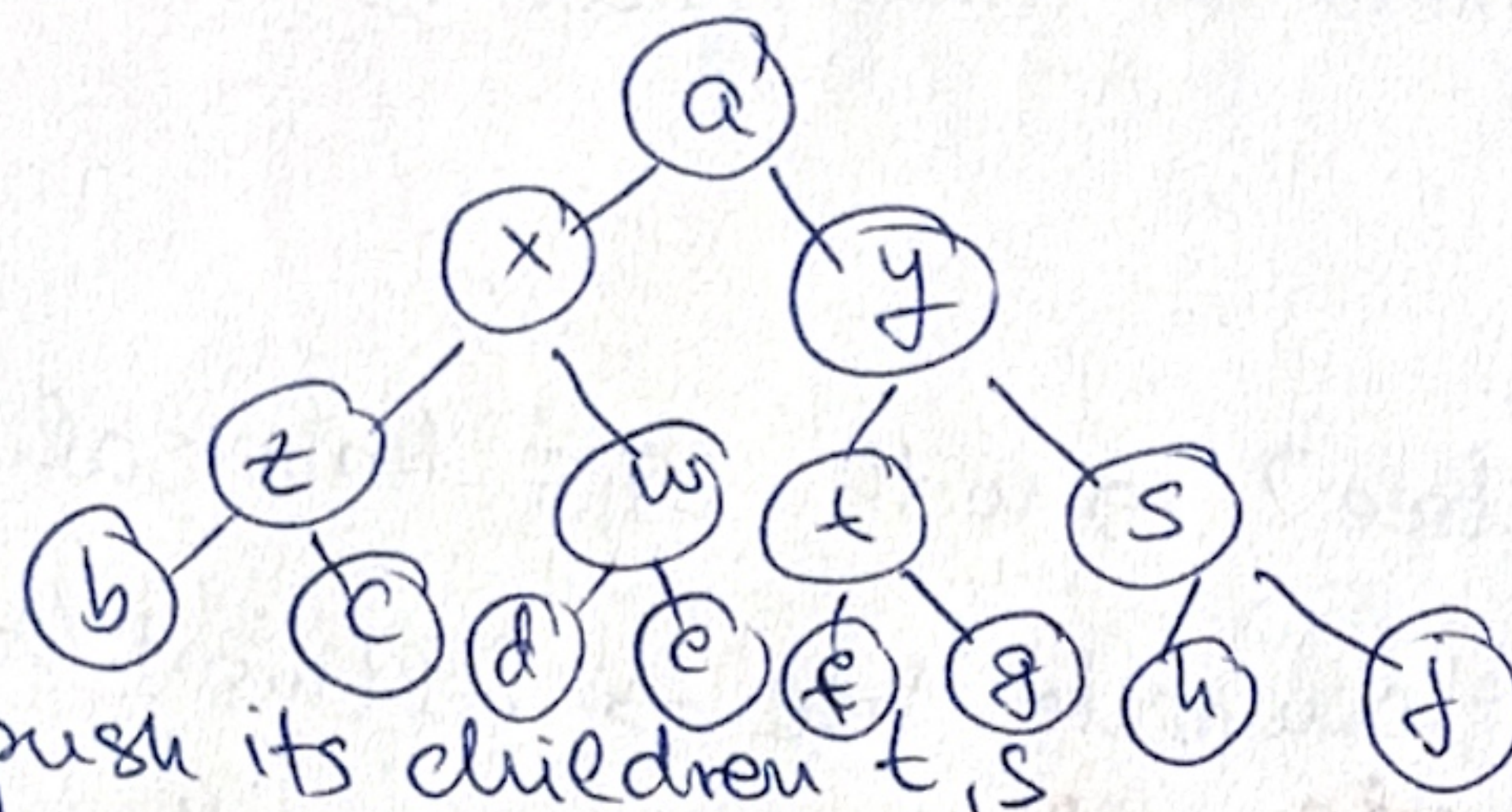
→ precompute how many nodes you have on the first K-1 levels, and count when you pass them (assume complete tree)

is not complete: $CL = \emptyset.1$

$nrNodesOnCL = 1 \cancel{0} \cancel{2} \cancel{1} \cancel{0} \cancel{4} \cancel{3} 2$

$nrChildren = \cancel{0} \cancel{1} \cancel{0} \cancel{2} \cancel{4} \cancel{0} \cancel{7} 4$

$Q: \emptyset, \cancel{x} \cancel{y} \cancel{z} wtsbcde$



pop node $y \rightarrow$ push its children t, s

$\rightarrow nrChildren: 2$

$\rightarrow nrNodesOnCL = 0 \Rightarrow$ done w current level.

$\rightarrow$ use two queues (to separate the two levels)

$CL: \emptyset \cancel{x} 2$

$Q_1: \emptyset \xrightarrow{\text{root}} \cancel{z} w \cancel{t} s$

$Q_2: \cancel{x} \cancel{y}$

How to know when a level is done?

easiest to implement $\rightarrow$ use one queue and a special value to denote end of a level.

$CL: \emptyset \cancel{x} 2$

$Q: \emptyset \cancel{NIL} \cancel{x} \cancel{y} \cancel{NIL} z w t s \cancel{NIL}$

Recursive $\rightarrow$ decrement k approach: (2 functions)

subalg $levelK(\underline{mode}, k): \rightarrow$ mode could be NIL.

if $mode \neq NIL$ then

if $k = 0$ then

| print([mode].info)

else

| levelK([mode].left, $k-1$)

| levelK([mode].right, $k-1$)

subalg printLevelK(bt, k):

| levelK(bt.root, k)

$\rightarrow$ "WRAPPER FUNCTION"

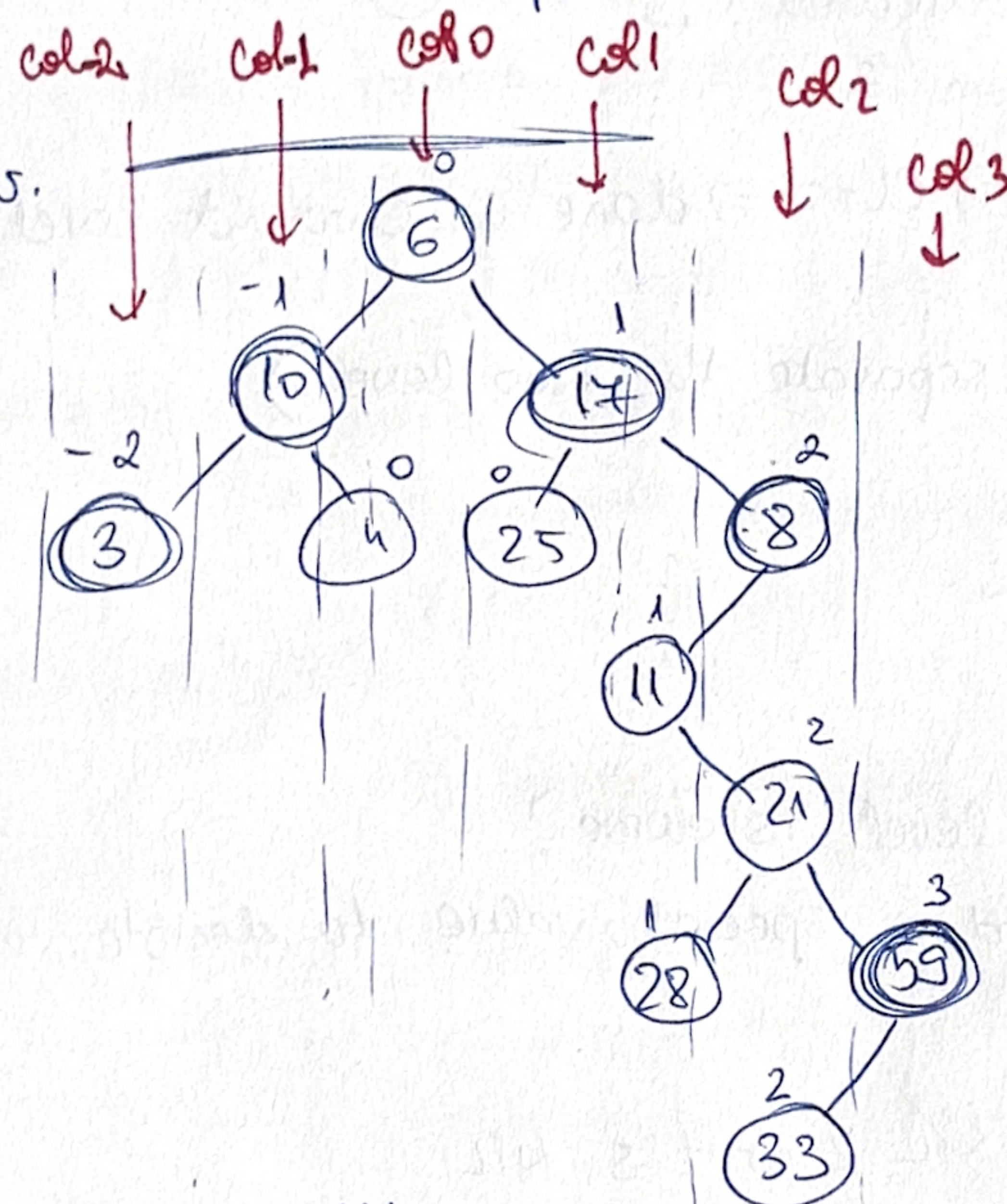
subalg printLevelK(bt, k)
 ↳ levelK(bt.root, k)

Complexity: $O(n)$ $\left\{ \begin{array}{l} \text{we need the min/max is with +} \\ O(2^k) \end{array} \right.$

How to compute height of a tree? $\rightarrow$ with DFS traversal.

↳ compute tree $\rightarrow$ leftmost/rightmost (take length of one path)

3) Print top view.



imagine they're laid on the ground.

top view: 3 10 6 14 8 59

For each column, print the first node that you find.

$\rightarrow$ Deque $\rightarrow$ double ended (to both sides)